

5MB单片大小

3并发文件

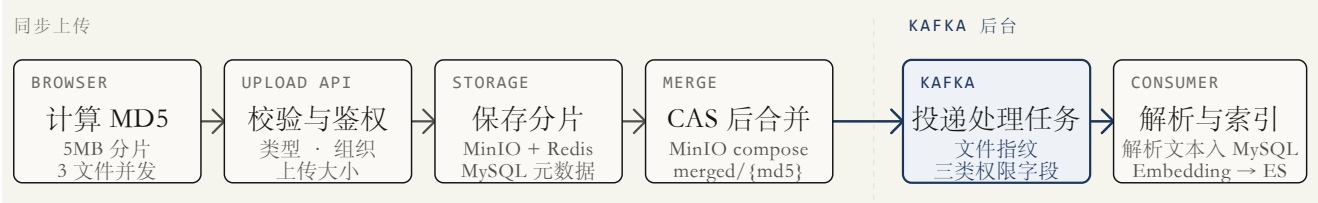
512切片上限

5X消费尝试

上传请求负责把文件可靠地放入 MinIO 并投递任务；解析、切片、Embedding 与 ES 建索引由 Kafka 消费者继续完成。权限字段 `userId`、`orgTag`、`isPublic` 从上传贯穿到检索文档。

端到端主链路

HTTP 很快返回，耗时建库留在后台



模块、步骤与源码坐标

按调用顺序阅读

模块	关键步骤	对应源码
前端任务	计算整文件 MD5；按 5MB 串行上传；同时处理最多 3 个文件	<code>common.ts:72-101</code> <code>knowledge-base/index.ts:8-184</code>
上传入口	首片校验类型；补主组织；校验大小；返回 Redis 位图进度	<code>UploadController.java:101-205, 230-279</code>
分片存储	写 MinIO <code>chunks/{md5}/{index}</code> ；Redis 位图记进度；MySQL 记分片元数据	<code>UploadService.java:70-211, 329-510</code>
合并门问	校验分片完整；CAS 执行 <code>UPLOADING</code> → <code>MERGING</code> ，挡住重复合并	<code>UploadController.java:289-377</code> <code>FileUploadRepository.java:79-84</code>
对象合并	MinIO compose 到 <code>merged/{md5}</code> ；清理分片与位图；状态置 <code>COMPLETED</code>	<code>UploadService.java:528-642</code>
任务投递	封装文件指纹、预签名 URL、文件名与权限；Kafka 事务发送	<code>UploadController.java:383-445</code> <code>FileProcessingTask.java:13-19</code>
异步解析	下载合并文件；PDFBox 按页并过滤重复页眉页脚；其他格式由 Tika 流式解析；语义切片入 MySQL	<code>FileProcessingConsumer.java:42-90</code> <code>ParseService.java:72-290, 315-544</code>
向量索引	读取文本切片；批量 Embedding；组装 <code>EsDocument</code> ；Bulk 写入 <code>knowledge_base</code> ；回写实际用量	<code>VectorizationService.java:47-125</code> <code>ElasticsearchService.java:34-69</code>

状态语义：`file_upload.COMPLETED` 只表示 MinIO 合并完成，前端此时也显示“已完成”；真正可检索要等消费者写完 `document_vectors` 与 `Elasticsearch`。失败每 3 秒重试一次，最多重试 4 次，耗尽进入 `file-processing-dlt`。